

To create a Blackjack deck, you would only need to supply a different dictionary for `rankValueDict`, like this:

```
blackJackDict = {'Ace':1, '2':2, '3':3, '4':4, '5':5,
                 '6':6, '7':7, '8': 8, '9':9, '10':10,
                 'Jack':10, 'Queen':10, 'King':10}
oBlackjackDeck = Deck(window, rankValueDict=blackJackDict)
```

Once you create the `oBlackjackDeck` this way, you can then call the existing `shuffle()` and `getCard()` methods with no change. In the implementation of Blackjack, you would also have to deal with the fact that an ace can have a value of 1 or 11. But that, as we say, is an exercise left to the reader!

Games with Unusual Card Decks

There are a number of card games that do not use a standard deck of 52 cards. The game of canasta requires at least two decks with jokers, for a total of 108 cards. A pinochle deck consists of two copies of 9, 10, jack, queen, king, and ace for each suit, for a total of 48 cards.

For games like these, you could still use the `Deck` class, but you would need to create a subclass with `Deck` as the base class. The new `CanastaDeck` or `PinochleDeck` class would need to have its own `__init__()` method that builds a deck as a list consisting of the appropriate `Card` objects. However, the `shuffle()` and `getCard()` methods could be inherited from the `Deck` class. Therefore, a `CanastaDeck` or `PinochleDeck` class would subclass the `Deck` class and consist of only an `__init__()` method.

Summary

In this chapter, we built a GUI version of the Higher or Lower card game from Chapter 1 using highly reusable `Deck` and `Card` classes. The main code instantiates a `Game` object, which creates a `Deck` object that instantiates 52 `Card` objects, one for each card in the resulting deck. Each `Card` object is responsible for drawing its appropriate image in the window and can respond to queries about its name, rank, suit, and value. The `Game` class, which contains the logic of the game, is separate from the main code, which runs the main loop.